CS3230

"Algorithms is Cool"

- ❑ Where do we use Algorithms
- ❑ Analysis of Algorithms & Why it is important
- ❑ Sample Computational Problems

Why Algorithm is Cool:

Algorithms is Anywhere & Everywhere.

Hon Wai Leong, NUS

© Leong Hon Wai, 2007-13

Motivation

- Where do we use Algorithms?
  - Computer Science, Engineering
  - Business, Operations Research
  - Finance, Social Sciences, … , Everywhere
- Why is Performance Important?
  - Exponential-size solution space
  - NP-completeness
- Problem Size Explosion
  - Computer Chip Complexity,
  - Database Sizes,
  - Human Genome Project, WWW (Google),

© Leong Hon Wai, 2007-13

Diverse Applications

- **Design the next generation computer chip**
  - ***how to design optimally (wrt speed, power, area)***
- **Internet (WWW)**
  - ***how to manage, manipulate large volume of data***
- **e-Commerce**
  - ***how to manage transaction (secure, private)***
- **Logistics**
  - ***how to manage transport/transfer of goods, people***
- **Human Genome Project**
  - ***how to analyze huge volume of DNA/protein data***

© Leong Hon Wai, 2007-13

Analysis of algorithms

The theoretical study of computer-program performance and resource usage.

What's often more important than *performance*?

- modularity
- correctness
- maintainability
- functionality
- robustness
- user-friendliness
- programmer time
- simplicity
- extensibility
- reliability

© Leong Hon Wai, 2007-13

Why study algorithms and performance?

- Algorithms help us to understand ***scalability***.
- Performance often draws the line between what is ***feasible*** and what is ***impossible***.
- Algorithmic mathematics provides a ***language*** for talking about program behavior.
- Performance is the ***currency*** of computing.
- The lessons of program performance ***generalize*** to other computing resources.
- Speed is ***fun***!

© Leong Hon Wai, 2007-13

Sample Combinatorial Problems

Combinatorial Problem	*Computational Complexity*
❖ Maintaining student records	*Easy*
❖ Data Compression	*Easy*
❖ Traveling Salesman Problem	*Hard*
❖ Shortest Route Planning	*Easy*
❖ Program Halting Problem	*Impossible*
❖ VLSI Chip Layout Problem	*Hard*
❖ Examination Time Table Scheduling	*Hard*
❖ Checking if graph is acyclic	*Easy*
❖ Computer Deadlock Problem	*Easy*
❖ Sorting records in a Database	*Easy*
❖ Finding patterns in a text	*Easy*

© Leong Hon Wai, 2007-13

Combinatorial Problems…

- **General Combinatorial Problem**
  - Given a finite, discrete set S of objects
  - ***To compute some function*** $f(S)$

- **Algorithmic Issues…**
  - Representation of the set S
  - Efficient manipulation of the set S
  - Efficient algorithm to compute $f(S)$

© Leong Hon Wai, 2007-13

Design and Analysis of Algorithms

- Given a problem *P*,
  - Can it be solved?

Computability

- If "Yes", given an algorithm *A* for solving *P*,
  - Is algorithm *A* correct ?
  - How good is algorithm *A* ?
  - Can find a better algorithm *A'* ?

Verification

Efficiency

- How do we define good?
  - How much time it takes.
  - How much space it uses.

Time Complexity

Space Complexity

© Leong Hon Wai, 2007-13

Complexity

- Given an algorithm *A* for problem *P*,
- How to do better?

  Complexity of Algorithm

  - Is the time complexity of *A* polynomial ?
  - Can we design faster algorithm *A'*?
  - Can we design algorithm *A''* that uses less space
- Can we do better?

  Complexity of Problem

  - Is the problem NP-complete?
  - Can we establish lower bounds
  - Is the algorithm "*best possible*"

© Leong Hon Wai, 2007-13

Thank you.

Q & A

NUS National University of Singapore

School of Computing

© Leong Hon Wai, 2007-13